Code Companion

A line-by-line guide to the csdh-postop-seizure-risk analysis code

Niels Pacheco-Barrios

Department of Neurosurgery, Beth Israel Deaconess Medical Center, Harvard Medical School

May 19, 2026

Each chapter explains one part of the codebase in plain English, then shows the actual code with line numbers and syntax highlighting, and then unpacks what each block is doing. You do not need to be fluent in Python to follow along; the Python primer in Chapter 1 introduces the constructs that recur throughout. If you only have an hour, read Chapter 1, Chapter 2 (`_shared.py`), and Chapter 6 (Firth + conformal). Those three pieces are the spine of the analysis.

Contents

1 A short Python primer for clinicians **2**

1.1 How a Python file is organised . . . 2
1.2 Variables, lists, and dictionaries . . . 2
1.3 Functions: code with a name and inputs . . . 2
1.4 Pandas, numpy, and scikit-learn . . . 3
1.5 Pipelines and the scikit-learn fit / predict pattern . . . 3
1.6 Cross-validation in three lines . . . 3

2 `_shared.py` — the foundations every script depends on **4**

2.1 Imports and global thread caps . . . 4
2.2 Path constants . . . 4
2.3 Feature lists . . . 4
2.4 The loader functions . . . 5
2.5 Pipeline factories . . . 6
2.6 Out-of-fold prediction helper . . . 6

3 `02_calibration.py` — measuring how trustworthy the probabilities are **8**

4 `04_loho.py` — leave-one-hospital-out validation **10**

5 `14_decision_tree.py` — the TreeAge-style cost-effectiveness model **13**

6 `24_firth_bayes_lr.py` — the deployment model **16**

7 `25_conformal_prediction.py` — turning probabilities into bedside decisions **20**

8 `16_voi_evpi.py` — value-of-information analysis **23**

9 `29_main_figures_jnnp.py` — publication-grade figures **26**

10 `27_build_jnnp_manuscript.py` — building the Word manuscript from code **29**

11 `30_export_calculator_assets.py` — packaging the model for the dashboard **31**

12 The interactive dashboards — HTML, CSS, JavaScript **34**

12.1 Anatomy of an HTML page . . . 34
12.2 How the risk calculator works . . . 34
12.3 How the savings calculator works . . . 35
12.4 How the interactive callgraph works . . . 36

13 The callgraph — how the scripts depend on each other **37**

14 Other scripts — what they do, in one paragraph each **38**

14.1 `03_dca.py` . . . 38
14.2 `05_temporal_leakage.py` . . . 38
14.3 `06_overfitting.py` . . . 38
14.4 `07_missing_data.py` . . . 38
14.5 `08_eicu_cohort.py` . . . 38
14.6 `09_competing_risks.py` . . . 38
14.7 `10_11_cea_pairwise.py` . . . 38
14.8 `12_nis_seizure_reclassify.py` . . . 38
14.9 `13_nis_grouped_lasso.py` . . . 38
14.10 `15_radiology_nlp.py` . . . 39
14.11 `17_build_slides.py` . . . 39
14.12 `18_bidmc_optimize.py` . . . 39
14.13 `19_transfer_learning.py` . . . 39
14.14 `20_build_manuscript.py` . . . 39
14.15 `21_imbalance_sweep.py` . . . 39
14.16 `22_diverse_stacking.py` . . . 39
14.17 `23_tabpfn_eval.py` . . . 39
14.18 `26_main_figures.py` . . . 39
14.19 `28_make_callgraph.py` . . . 39
14.20 `31_export_callgraph_json.py` . . . 39
14.21 `32_build_code_companion.py` . . . 39
14.22 `33_build_code_companion_pdf.py` . . . 40

15 Further study — Claude skills that help you learn the code **41**

1 A short Python primer for clinicians

Python is a programming language often described as 'executable pseudocode' because the constructs in a Python script tend to translate cleanly into English sentences. This chapter introduces the small set of building blocks that recur throughout our analysis code.

1.1 How a Python file is organised

A Python file (extension .py) is read from top to bottom. The topmost few lines state the purpose of the file in a triple-quoted docstring, then import the libraries the file will need.

```
"""Task 24 — Firth penalized + Bayesian logistic regression.
A docstring describes what this file does and what it produces.
"""
import os, sys
import numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression
from _shared import load_bidmc, RES, FIG, SEED
```

Lines starting with the word import bring in code from elsewhere. import numpy as np gives us a shorthand: from this line on, np stands for the numpy library. from sklearn.linear_model import LogisticRegression pulls one specific tool out of a larger library — exactly like importing one drug class from a pharmacopoeia.

1.2 Variables, lists, and dictionaries

```
# A single value
age = 73
prevalence = 0.073
cohort_name = "BIDMC"

# A list — an ordered collection
features = ["age", "sex", "preop_gcs", "sdh_thickness"]
features.append("mid_shift")

# A dictionary — a mapping from keys to values
feature_means = {"age": 73, "sex": 0.68, "preop_gcs": 14.1}
feature_means["epilepsy_hx"] = 0.02
```

Variables are labels that point to values. Lists hold ordered collections (square brackets). Dictionaries map a key to a value (curly braces). The hash sign # introduces a comment.

1.3 Functions: code with a name and inputs

```
def bootstrap_auc(y, p, n_boot=1000, seed=42):
    """Compute the AUC and a 95% bootstrap confidence interval."""
    rng = np.random.default_rng(seed)
    bs = []
    for _ in range(n_boot):
        idx = rng.integers(0, len(y), len(y))
```

```
        bs.append(roc_auc_score(y[idx], p[idx]))
    lo, hi = np.percentile(bs, [2.5, 97.5])
    return float(roc_auc_score(y, p)), float(lo), float(hi)
```

A function is a named recipe. def starts the definition; the name follows; the inputs go inside the parentheses. Inputs can have default values. The indented block under def is the body. The for line is a loop. The return statement at the end is what the function gives back to its caller.

1.4 Pandas, numpy, and scikit-learn

Three external libraries do almost all of the analytical work in this codebase. numpy provides fast numerical arrays. pandas provides DataFrames (tables with named columns and labelled rows). scikit-learn (imported as sklearn) provides the machine-learning models, cross-validation splitters, and the standardisation machinery.

1.5 Pipelines and the scikit-learn fit / predict pattern

```
pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler",  StandardScaler()),
    ("classifier", FirthLogisticRegression()),
])
pipe.fit(X_train, y_train)
probs = pipe.predict_proba(X_test)[:, 1]
```

A scikit-learn pipeline is a chain of steps. When we call fit, the pipeline runs imputation on the training data, then standardisation, then trains the classifier. When we call predict_proba, the pipeline runs the same imputation and standardisation (using the training-time parameters) and then asks the classifier for predicted probabilities.

> A pipeline guarantees that the same preprocessing applied to the training data is applied — using the training-time statistics — to any new data. Without a pipeline it is easy to compute the column means on the test set and introduce a leakage bias.

1.6 Cross-validation in three lines

```
rskf = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=42)
for train_idx, test_idx in rskf.split(X, y):
    pipe.fit(X.iloc[train_idx], y.iloc[train_idx])
    p_oof[test_idx] = pipe.predict_proba(X.iloc[test_idx])[:, 1]
```

RepeatedStratifiedKFold yields pairs of (train_idx, test_idx). Each pair is one fold: train on 80% of the data, evaluate on the held-out 20%, save the predicted probabilities for the held-out patients into p_oof (out-of-fold). After all folds are processed, p_oof contains one prediction for every patient.

2 `_shared.py` — the foundations every script depends on

_shared.py is the file every other script in the project imports. It defines path constants, feature lists, loader functions, and pipeline factories. Reading this file is the single highest-yield investment of your time, because the patterns it establishes appear in every other script.

2.1 Imports and global thread caps

The file opens with environment-variable settings that force the underlying numerical libraries (OpenBLAS, MKL, OpenMP) to use a single CPU thread per process. On Apple Silicon, multi-threaded scikit-learn can deadlock; setting n_jobs=1 throughout the codebase prevents that.

```
"""Shared utilities for revision analyses.

n_jobs is forced to 1 throughout for Apple Silicon stability.
"""
import os, sys, warnings, json
from pathlib import Path
import numpy as np
import pandas as pd
import joblib

warnings.filterwarnings("ignore")

ROOT = Path(__file__).resolve().parents[2]
OUT = ROOT / "revision_analyses"
RES = OUT / "results"
FIG = OUT / "figures"
CACHE = OUT / "cache"
for p in (RES, FIG, CACHE):
    p.mkdir(parents=True, exist_ok=True)

SEED = 42
N_JOBS = 1  # Apple Silicon stability — never change

... (246 more lines elided — see full script in the repository) ...
```

2.2 Path constants

ROOT is the project's top-level folder. OUT is revision_analyses/, RES is its results/ subfolder, FIG is figures/, CACHE is for intermediate arrays. The for loop creates each folder if missing.

2.3 Feature lists

POSTOP_A_FEATURES is a list of 21 column names. Defining it here, once, means every script that fits postop_A uses exactly the same 21 predictors. POSTOP_B_FEATURES is the same list with the three leakage-suspect variables removed — the temporal-leakage robustness analysis.

```
POSTOP_A_FEATURES = [
    "sex", "age", "blood_type", "sdh_type", "sdh_thickness", "csdh_size_change",
```

```
    "mid_shift", "hematoma_lat", "collection_density", "preop_gcs",
    "epilepsy_hx", "num_prev_sdh", "demographic", "procedures",
    "surg_decompression", "mma_embo", "drainage", "postop_gcs",
    "aed_timing_recoded", "prop_aed", "ab_eeg",
]
POSTOP_B_FEATURES = [
    "sex", "age", "blood_type", "sdh_type", "sdh_thickness", "csdh_size_change",
    "mid_shift", "hematoma_lat", "collection_density", "preop_gcs",
    "epilepsy_hx", "num_prev_sdh", "demographic", "procedures",
    "surg_decompression", "mma_embo", "drainage", "postop_gcs",
]
PREOP_FEATURES = [
    "sex", "age", "blood_type", "sdh_type", "sdh_thickness", "csdh_size_change",
    "mid_shift", "hematoma_lat", "collection_density", "preop_gcs",
    "epilepsy_hx", "num_prev_sdh", "demographic", "procedures",
]

def load_bidmc():
    df = pd.read_csv(BIDMC_CSV)
    df["aed_timing_recoded"] = df["aed_timing"].replace({2: 1})
    return df

... (217 more lines elided — see full script in the repository) ...
```

2.4 The loader functions

load_bidmc reads the cleaned BIDMC CSV and applies a single recoding. load_eicu does the same for eICU. load_eicu_pure adds an extra filter for the pure post-craniotomy sensitivity cohort.

```
def load_bidmc():
    df = pd.read_csv(BIDMC_CSV)
    df["aed_timing_recoded"] = df["aed_timing"].replace({2: 1})
    return df

# ──────────────────────────────────────────────────────────────────────
# eICU
# ──────────────────────────────────────────────────────────────────────
EICU_CSV = ROOT / "eicu_csdh_cohort_final.csv"

EICU_SET_A = ["age", "sex", "gcs_admission", "prior_seizures", "craniotomy",
              "burr_hole", "prophylactic_aed", "pre_admission_aed"]

EICU_SET_B_ADD = ["apache_score", "apache_gcs", "any_anticoagulant", "any_antiplatelet",
                  "any_steroid", "mechanical_ventilation", "icp_monitor", "blood_transfusion",
                  "comorbidity_count", "hx_hypertension", "hx_diabetes", "hx_stroke",
                  "hx_dementia", "hx_afib", "hx_alcohol", "pupil_abnormality",
                  "icp_available", "n_labs_24h", "n_vitals_24h", "Hgb_first",
                  "PT___INR_first", "WBC_x_1000_first", "albumin_first", "calcium_first",
                  "creatinine_first", "glucose_first", "lactate_first", "magnesium_first",
                  "platelets_x_1000_first", "potassium_first", "sodium_first",
                  "total_bilirubin_first", "temperature_mean", "sao2_mean",
                  "heartrate_mean", "respiration_mean", "systemicsystolic_mean",
                  "systemicdiastolic_mean", "systemicmean_mean"]
```

```

EICU_SET_C_ADD = ["gcs_24h", "gcs_min_24h", "gcs_max_24h", "gcs_delta", "n_gcs_assessments",
                  "sodium_slope_48h", "sodium_delta_48h", "sodium_cv_48h",
                  "potassium_slope_48h", "potassium_delta_48h", "potassium_cv_48h",
                  "glucose_slope_48h", "glucose_delta_48h", "glucose_cv_48h",
                  "creatinine_slope_48h", "creatinine_delta_48h", "creatinine_cv_48h",
                  "platelets_x_1000_slope_48h", "platelets_x_1000_delta_48h",
    "platelets_x_1000_cv_48h",
                  "lactate_slope_48h", "lactate_delta_48h", "lactate_cv_48h",
                  "WBC_x_1000_slope_48h", "WBC_x_1000_delta_48h", "WBC_x_1000_cv_48h",
                  "Hgb_slope_48h", "Hgb_delta_48h", "Hgb_cv_48h",
                  "heartrate_slope_48h", "heartrate_delta_48h", "heartrate_cv_48h",
                  "systemicsystolic_slope_48h", "systemicsystolic_delta_48h",
    "systemicsystolic_cv_48h",
                  "systemicmean_slope_48h", "systemicmean_delta_48h", "systemicmean_cv_48h",
                  "temperature_slope_48h", "temperature_delta_48h", "temperature_cv_48h",
                  "sao2_slope_48h", "sao2_delta_48h", "sao2_cv_48h",
                  "temperature_min", "temperature_max", "temperature_std",
... (182 more lines elided — see full script in the repository) ...
```

2.5 Pipeline factories

Each pipeline factory returns a fresh, untrained scikit-learn Pipeline. Returning a fresh pipeline matters because pipelines are mutable; fitting one in a loop would carry over state from the previous iteration.

```
def make_pipeline_postopA():
    from sklearn.pipeline import Pipeline
    from sklearn.compose import ColumnTransformer
    from sklearn.impute import SimpleImputer
    from sklearn.preprocessing import StandardScaler
    from imblearn.ensemble import BalancedRandomForestClassifier
    prep = ColumnTransformer(
        [("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                           ("sc",  StandardScaler())]), POSTOP_A_FEATURES)]
    )
    clf = BalancedRandomForestClassifier(
        n_estimators=300, min_samples_leaf=2,
        n_jobs=N_JOBS, random_state=SEED,
    )
    return Pipeline([("prep", prep), ("clf", clf)])
```

2.6 Out-of-fold prediction helper

```
def oof_predictions(make_pipe_fn, X, y, n_splits=5, n_repeats=5, groups=None):
    """Return mean OOF probability for each row across repeats."""
    from sklearn.model_selection import RepeatedStratifiedKFold
    from sklearn.base import clone
    rskf = RepeatedStratifiedKFold(n_splits=n_splits, n_repeats=n_repeats, random_state=SEED)
    p_acc = np.zeros(len(X))
    n_acc = np.zeros(len(X))
    for split_idx, (tr, te) in enumerate(rskf.split(X, y)):
```

```
        pipe = make_pipe_fn()
        pipe.fit(X.iloc[tr], y.iloc[tr])
        p = pipe.predict_proba(X.iloc[te])[:, 1]
        p_acc[te] += p
        n_acc[te] += 1
    return p_acc / np.maximum(n_acc, 1)
```

This function returns one predicted probability per patient by running repeated stratified K-fold cross-validation and averaging the predictions whenever a patient appears in multiple test folds. The accumulator trick — adding into p_acc and incrementing n_acc — is the standard idiom for averaging without storing every individual prediction.

3 `02_calibration.py` — measuring how trustworthy the probabilities are

AUC measures rank-order discrimination — does the model assign higher probabilities to patients who go on to seize? Calibration measures whether those probabilities are themselves trustworthy — when the model says 10%, do roughly 10% of those patients actually seize? Calibration is the metric that shapes clinical decisions.

```
"""Task 2 — Calibration analysis (BIDMC + eICU).

Outputs:
  results/02_calibration_metrics.csv
  figures/02_calibration_curves.png
  cache/oof_<model>.npz   (reused by DCA / temporal-leakage / etc.)
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

import numpy as np, pandas as pd, json
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from _shared import (
    load_bidmc, load_eicu, load_eicu_pure,
    POSTOP_A_FEATURES, POSTOP_B_FEATURES, PREOP_FEATURES,
    EICU_SET_A, EICU_SET_B, EICU_SET_C,
    make_pipeline_postopA, make_pipeline_postopB, make_pipeline_preop,
    make_pipeline_eicu, oof_predictions, calibration_metrics,
    RES, FIG, CACHE, SEED,
)

def bootstrap_calibration_ci(y, p, n_boot=1000, seed=42):
    """Fix F: paired-bootstrap 95% CI for Brier, CITL, slope/intercept, AUC."""
    from sklearn.metrics import brier_score_loss, roc_auc_score
    from sklearn.linear_model import LogisticRegression
    rng = np.random.default_rng(seed)
    n = len(y); collect = {k: [] for k in ("brier", "citl", "slope", "intercept", "auc")}
    for _ in range(n_boot):
        idx = rng.integers(0, n, n)
        yb, pb = y[idx], p[idx]
        if len(np.unique(yb)) < 2: continue
        # Brier
        collect["brier"].append(brier_score_loss(yb, pb))
        # CITL = observed − mean predicted
        collect["citl"].append(yb.mean() - pb.mean())
        # AUC
        collect["auc"].append(roc_auc_score(yb, pb))
        # slope / intercept via logit-regression of outcome on logit(pred)
        p_safe = np.clip(pb, 1e-6, 1 - 1e-6)
        x_logit = np.log(p_safe / (1 - p_safe)).reshape(-1, 1)
        try:
```

```
            lr = LogisticRegression(C=1e6, solver="lbfgs", max_iter=2000).fit(x_logit, yb)
            collect["intercept"].append(float(lr.intercept_[0]))
            collect["slope"].append(float(lr.coef_[0, 0]))
        except Exception:
            pass
    ci = {}
    for k, v in collect.items():
        if v:
            lo, hi = np.percentile(v, [2.5, 97.5])
            ci[f"{k}_lo"] = float(lo); ci[f"{k}_hi"] = float(hi)
        else:
            ci[f"{k}_lo"] = float("nan"); ci[f"{k}_hi"] = float("nan")
    return ci

def run_one(name, make_pipe, X, y, n_splits=5, n_repeats=5):
    print(f"  {name}: n={len(y)}, events={int(y.sum())}, splits={n_splits}x{n_repeats}",
    flush=True)
    p = oof_predictions(make_pipe, X, y, n_splits=n_splits, n_repeats=n_repeats)
    np.savez(CACHE / f"oof_{name}.npz", y=y.values, p=p)
    m = calibration_metrics(y.values, p)
    m["model"] = name
    m["n"] = int(len(y))
    m["events"] = int(y.sum())
    # Fix F: bootstrap 95% CIs on the core calibration metrics
    m.update(bootstrap_calibration_ci(y.values, p, n_boot=1000, seed=42))
    return m, p

def main():
    rows = []
    cohorts = {}

    # BIDMC ──────────────────────────────────────────────
    df = load_bidmc()
    y = df["seizure"].astype(int)
    print(f"\nBIDMC n={len(df)}, events={y.sum()}")
    m, p = run_one("bidmc_postopA",
... (90 more lines elided — see full script in the repository) ...
```

The metrics are computed by calibration_metrics in _shared.py: Brier score, calibration-in-the-large, calibration slope, intercept, ECE, MCE, and the Hosmer-Lemeshow chi-square. A well-calibrated model has Brier near p×(1−p), CITL near zero, slope near one, and intercept near zero.

4 `04_loho.py` — leave-one-hospital-out validation

External validation in eICU is more demanding than ordinary cross-validation because it asks: does the model generalise from hospital A to hospital B? We answer that by leaving one hospital out at a time, training on the remaining 138, and scoring the held-out hospital.

```python
"""Task 4 — Leave-one-hospital-out CV on eICU.

For each hospital with >= MIN_EVENTS seizures, train on remaining 138 hospitals
and evaluate on the held-out hospital. Report AUC distribution.

Outputs:
  results/04_loho_per_hospital.csv
  results/04_loho_summary.csv
  figures/04_loho_distribution.png
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"

import numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score, average_precision_score, brier_score_loss
from _shared import (
    load_eicu, load_eicu_pure, EICU_SET_A, EICU_SET_C,
    make_pipeline_eicu, RES, FIG, CACHE, SEED,
)

MIN_EVENTS = 3   # held-out hospital must have ≥3 seizures for AUC
MIN_PATIENTS = 10

def make_pipe_light(features):
    """Lighter pipeline (200 trees) for LOHO speed."""
    from sklearn.pipeline import Pipeline
    from sklearn.compose import ColumnTransformer
    from sklearn.impute import SimpleImputer
    from sklearn.preprocessing import StandardScaler
    from sklearn.ensemble import RandomForestClassifier
    prep = ColumnTransformer(
        [("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                           ("sc",  StandardScaler())]), features)]
    )
    clf = RandomForestClassifier(
        n_estimators=200, min_samples_leaf=2, class_weight="balanced",
        n_jobs=1, random_state=SEED,
    )
    return Pipeline([("prep", prep), ("clf", clf)])

def loho_for(df, features, set_name, cohort_name, ckpt_path):
    """Returns a DataFrame; supports incremental checkpointing."""
```

```
    y = df["seizure"].astype(int).values
    H = df["hospitalid"].values
    X = df[features]
    hospitals = sorted(df["hospitalid"].unique())
    print(f"  {cohort_name} / {set_name}: {len(hospitals)} hospitals, n={len(df)},
     sz={y.sum()}", flush=True)

    if ckpt_path.exists():
        rows = pd.read_csv(ckpt_path).to_dict("records")
        done = {r["hospitalid"] for r in rows
                if r["cohort"] == cohort_name and r["set"] == set_name}
        print(f"    resuming, {len(done)} hospitals done", flush=True)
    else:
        rows = []; done = set()

    for i, h in enumerate(hospitals):
        if int(h) in done:
            continue
        te = (H == h)
        tr = ~te
        if y[te].sum() < MIN_EVENTS or te.sum() < MIN_PATIENTS:
            continue
        if y[tr].sum() == 0 or (1 - y[tr]).sum() == 0:
            continue
        pipe = make_pipe_light(features)
        pipe.fit(X[tr], y[tr])
        p = pipe.predict_proba(X[te])[:, 1]
        auc = roc_auc_score(y[te], p)
        prauc = average_precision_score(y[te], p)
        brier = brier_score_loss(y[te], p)
        rows.append({
            "cohort": cohort_name, "set": set_name, "hospitalid": int(h),
            "n_test": int(te.sum()), "events_test": int(y[te].sum()),
            "auc": auc, "prauc": prauc, "brier": brier,
        })
        # incremental checkpoint
        pd.DataFrame(rows).to_csv(ckpt_path, index=False)
        if len(rows) % 5 == 0:
            print(f"    {len(rows)} hospitals scored, last AUC={auc:.3f}", flush=True)
    return pd.DataFrame(rows)

def main():
    df_full = load_eicu()
    df_pure = load_eicu_pure()
    print(f"Full: n={len(df_full)} events={df_full.seizure.sum()}
     hospitals={df_full.hospitalid.nunique()}")
    print(f"Pure: n={len(df_pure)} events={df_pure.seizure.sum()}
     hospitals={df_pure.hospitalid.nunique()}")

    ckpt = CACHE / "04_loho_ckpt.csv"
    parts = []
    parts.append(loho_for(df_full, EICU_SET_A, "Set_A", "full", ckpt))
    parts.append(loho_for(df_full, EICU_SET_C, "Set_C", "full", ckpt))
    parts.append(loho_for(df_pure, EICU_SET_C, "Set_C", "pure", ckpt))
```

```
    perh = pd.concat(parts, ignore_index=True).drop_duplicates(["cohort","set","hospitalid"])
    perh.to_csv(RES / "04_loho_per_hospital.csv", index=False)

    # ── Fix D: Random-effects meta-analytic pooling (DerSimonianLaird)
    # Treat each held-out hospital AUC as one study; transform to logit-AUC,
    # compute within-study variance via HanleyMcNeil, then DL estimator for
    # between-hospital heterogeneity τ².
    def hanley_mcneil_var(auc, n1, n0):
        """Approximate variance of AUC (Hanley & McNeil 1982)."""
        auc = float(np.clip(auc, 1e-6, 1 - 1e-6))
        n1 = max(int(n1), 1); n0 = max(int(n0), 1)
        Q1 = auc / (2.0 - auc); Q2 = 2.0 * auc**2 / (1.0 + auc)
        return (auc * (1 - auc)
                + (n1 - 1) * (Q1 - auc**2)
                + (n0 - 1) * (Q2 - auc**2)) / (n1 * n0)

    def logit(p):  return np.log(p / (1 - p))
    def invlogit(x): return 1.0 / (1.0 + np.exp(-x))

    def random_effects_pool(g):
        """DerSimonianLaird pooling on logit-AUC scale."""
        auc = g["auc"].values
... (104 more lines elided — see full script in the repository) ...
```

loho_for handles the leave-one-hospital-out loop and writes a CSV row per hospital. random_effects_pool turns those per-hospital AUCs into a single meta-analytic estimate using the DerSimonian-Laird method.

5 `14_decision_tree.py` — the TreeAge-style cost-effectiveness model

Decision-tree models in health economics compute the expected cost and expected QALYs of a strategy by enumerating every possible patient trajectory, weighting each by its probability, and adding the results. This script implements that for our four strategies and renders Figure 4.

```python
"""Task 14 — TreeAge-style decision tree (Figure 4 for CEA).

Renders the 4-strategy decision tree with:
  □ decision node (root)
  ○ chance nodes (seizure y/n, detected y/n)
  ▷ terminal nodes (cost / QALY pair)

Uses the deterministic point-estimate parameters from 10_11_cea_pairwise.py and
performs expected-value rollback at each chance node so the tree shows both the
structure and the rolled-up E[Cost], E[QALY] per strategy.

Outputs:
  figures/14_decision_tree.{png,pdf,svg}
  results/14_decision_tree_rollback.csv
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
os.environ["OMP_NUM_THREADS"] = "1"

import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.patches import FancyBboxPatch
import pandas as pd

from _shared import RES, FIG

# ── Base-case parameters (point estimates from Params in 10_11_cea_pairwise.py)
# Kept here as plain dict for readability in the figure.
P = dict(
    p_seizure_base   = 0.12,
    aed_rrr          = 0.45,
    model_sens       = 0.842,
    model_spec       = 0.504,
    p_detect_ceeg    = 0.95,
    p_detect_clin    = 0.30,
    cost_ml          = 50.0,
    cost_aed_total   = 200.0 + 1507.0 + 441.0,
    cost_aed_total_ceeg = 200.0 + 1293.0 + 140.0,
    cost_ceeg_total  = 1500.0 * 3,
    cost_sz_detected = 2500.0 + (20.3 - 9.8) * 3500.0 + 1000.0,
    cost_sz_undet_mult = 1.3,
    qaly_baseline    = 0.75,
    qaly_sz_loss     = 0.10,
    qaly_aed_loss    = 0.02 * (66 / 365),
    qaly_aed_loss_ceeg = 0.02 * (21 / 365),
    horizon_qaly_no_sz = 7.5,   # ~10y * 0.75 discounted
```

```
    horizon_qaly_sz    = 6.8,
    horizon_qaly_undet = 6.3,
)

# ── Strategy logic — point-estimate rollback ──────────────────────────
def rollback_strategy(name):
    """Return list of leaves: (path_label, prob, cost, qaly)."""
    p_sz = P["p_seizure_base"]
    leaves = []

    if name == "obs":
        # No prophylaxis, clinical detection only
        p_det = P["p_detect_clin"]
        leaves.append((f"→szdet",  p_sz * p_det,
                       P["cost_sz_detected"],
                       P["horizon_qaly_sz"] - P["qaly_sz_loss"]))
        leaves.append((f"→szmiss", p_sz * (1 - p_det),
                       P["cost_sz_detected"] * P["cost_sz_undet_mult"],
                       P["horizon_qaly_undet"] - P["qaly_sz_loss"]))
        leaves.append((f"no sz",    1 - p_sz, 0.0,
                       P["horizon_qaly_no_sz"]))

    elif name == "aed_all":
        # Universal AED, clinical detection
        p_sz_treated = p_sz * (1 - P["aed_rrr"])
        p_det = P["p_detect_clin"]
        leaves.append(("→→AEDszdet",  p_sz_treated * p_det,
                       P["cost_aed_total"] + P["cost_sz_detected"],
                       P["horizon_qaly_sz"] - P["qaly_sz_loss"] - P["qaly_aed_loss"]))
        leaves.append(("→→AEDszmiss", p_sz_treated * (1 - p_det),
                       P["cost_aed_total"] + P["cost_sz_detected"] * P["cost_sz_undet_mult"],
                       P["horizon_qaly_undet"] - P["qaly_sz_loss"] - P["qaly_aed_loss"]))
        leaves.append(("→AEDno sz",   1 - p_sz_treated,
                       P["cost_aed_total"],
                       P["horizon_qaly_no_sz"] - P["qaly_aed_loss"]))

    elif name == "ml_aed":
        # ML → AED if predicted high risk; clinical detection
        sens = P["model_sens"]; spec = P["model_spec"]
        tp = sens * p_sz; fn = (1 - sens) * p_sz
        fp = (1 - spec) * (1 - p_sz); tn = spec * (1 - p_sz)
        rrr = P["aed_rrr"]
        # TP: AED reduces seizure risk by rrr; if seizes, detected clinically
        leaves.append(("ML→+→TPprevent", tp * rrr,
                       P["cost_ml"] + P["cost_aed_total"],
                       P["horizon_qaly_no_sz"] - P["qaly_aed_loss"]))
        leaves.append(("ML→+→TPsz",     tp * (1 - rrr),
                       P["cost_ml"] + P["cost_aed_total"] + P["cost_sz_detected"],
                       P["horizon_qaly_sz"] - P["qaly_sz_loss"] - P["qaly_aed_loss"]))
        # FP: AED but never would have seized
        leaves.append(("ML→+FP",        fp,
                       P["cost_ml"] + P["cost_aed_total"],
                       P["horizon_qaly_no_sz"] - P["qaly_aed_loss"]))
        # FN: no AED, seizure occurs, clinical detection
```

```
        p_det = P["p_detect_clin"]
        leaves.append(("→→MLFNdet",   fn * p_det,
                       P["cost_ml"] + P["cost_sz_detected"],
                       P["horizon_qaly_sz"] - P["qaly_sz_loss"]))
        leaves.append(("→→MLFNmiss",  fn * (1 - p_det),
                       P["cost_ml"] + P["cost_sz_detected"] * P["cost_sz_undet_mult"],
                       P["horizon_qaly_undet"] - P["qaly_sz_loss"]))
        # TN: no AED, no seizure
        leaves.append(("→MLTN",        tn,
                       P["cost_ml"],
                       P["horizon_qaly_no_sz"]))

    elif name == "ml_ceeg":
        # ML → cEEG monitoring if predicted high risk
        sens = P["model_sens"]; spec = P["model_spec"]
        tp = sens * p_sz; fn = (1 - sens) * p_sz
        fp = (1 - spec) * (1 - p_sz); tn = spec * (1 - p_sz)
        p_det = P["p_detect_ceeg"]   # cEEG much higher detection
        # High-risk branch: cEEG + targeted AED (lower-burden)
... (188 more lines elided — see full script in the repository) ...
```

rollback_strategy returns the list of terminal leaves. compute_ev does the rollback: probability times payoff, summed. render_tree draws the tree in matplotlib using the TreeAge visual convention.

6 `24_firth_bayes_lr.py` — the deployment model

With 48 events in BIDMC, ordinary maximum-likelihood logistic regression is unstable. Firth's penalty stabilises the coefficient estimates and produces valid confidence intervals even with rare events.

```
"""Task 24 — Firth penalized LR + Bayesian LR with priors derived from eICU.

For rare-event clinical prediction (48/655), maximum-likelihood logistic regression
suffers from separation bias and inflated coefficient SEs. Two principled fixes:

  (a) Firth (1993) penalized likelihood logistic regression
      - Adds Jeffreys-prior penalty: □ _Fβ() = □ β() + 0.5 log|Iβ()|
      - Demonstrated to stabilize estimation when events_per_variable < 10
      - Puhr et al., Stat Med 2017 (arXiv:2101.07620)

  (b) Bayesian logistic regression with informative priors derived from the
      n=5,376 eICU cohort
      - Step 1: fit elastic-net LR on eICU using overlapping shared features
      - Step 2: use eICU coefficient point estimates as Gaussian prior means
        on the corresponding BIDMC coefficients
      - Step 3: posterior mode by penalized IRLS with diagonal Gaussian prior
        precision (acts as L2 with non-zero center)
      - This is a principled alternative to "transfer learning" that does not
        require shared feature names beyond the prior-informed subset.

Evaluation: 5×5 repeated stratified CV; bootstrap 95% AUC CI; paired DeLong
test vs the BalancedRandomForest baseline.

Outputs:
  results/24_firth_bayes_lr.csv
  figures/24_firth_bayes_lr.{png,pdf}
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
os.environ["OMP_NUM_THREADS"] = "1"

import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.metrics import roc_auc_score, brier_score_loss, average_precision_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from imblearn.ensemble import BalancedRandomForestClassifier

from firthlogist import FirthLogisticRegression

from _shared import (load_bidmc, load_eicu, POSTOP_A_FEATURES, POSTOP_B_FEATURES,
                     RES, FIG, CACHE, SEED)
```

```
N_SPLITS, N_REPEATS = 5, 5

# Shared features for eICU prior derivation (match BIDMC names → eICU names)
SHARED_BIDMC_NAMES = ["age", "sex", "preop_gcs", "epilepsy_hx", "prop_aed"]
SHARED_EICU_NAMES  = ["age", "sex", "gcs_admission", "prior_seizures", "prophylactic_aed"]

def make_prep(features):
    return ColumnTransformer([
        ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                          ("sc",  StandardScaler())]), features)])

def cv_oof(make_pipe_fn, X, y, n_splits=N_SPLITS, n_repeats=N_REPEATS):
    rskf = RepeatedStratifiedKFold(n_splits=n_splits, n_repeats=n_repeats, random_state=SEED)
    p_acc = np.zeros(len(X)); n_acc = np.zeros(len(X))
    for tr, te in rskf.split(X, y):
        try:
            p = make_pipe_fn()
            p.fit(X.iloc[tr], y.iloc[tr])
            prob = p.predict_proba(X.iloc[te])[:, 1]
            p_acc[te] += prob; n_acc[te] += 1
        except Exception:
            continue
    return p_acc / np.maximum(n_acc, 1)

def bootstrap_auc(y, p, n_boot=1000, seed=SEED):
    rng = np.random.default_rng(seed); bs = []
    n = len(y)
    for _ in range(n_boot):
        idx = rng.integers(0, n, n)
        if len(np.unique(y[idx])) < 2: continue
        bs.append(roc_auc_score(y[idx], p[idx]))
    lo, hi = np.percentile(bs, [2.5, 97.5]) if bs else (np.nan, np.nan)
    return float(roc_auc_score(y, p)), float(lo), float(hi)

def delong_test(y, p1, p2):
    y = np.asarray(y); p1 = np.asarray(p1); p2 = np.asarray(p2)
    pos = (y == 1); neg = (y == 0)
    m, n = pos.sum(), neg.sum()
    if m < 2 or n < 2: return float("nan"), float("nan")
    def struct(s):
        sp = s[pos]; sn = s[neg]
        V10 = np.array([(np.sum(sn < v) + 0.5 * np.sum(sn == v)) / n for v in sp])
        V01 = np.array([(np.sum(sp > v) + 0.5 * np.sum(sp == v)) / m for v in sn])
        return V10.mean(), V10, V01
    a1, v10_1, v01_1 = struct(p1); a2, v10_2, v01_2 = struct(p2)
    s10 = (np.var(v10_1, ddof=1) + np.var(v10_2, ddof=1)
           - 2 * np.cov(v10_1, v10_2, ddof=1)[0,1])
    s01 = (np.var(v01_1, ddof=1) + np.var(v01_2, ddof=1)
           - 2 * np.cov(v01_1, v01_2, ddof=1)[0,1])
    var_diff = s10 / m + s01 / n
    if var_diff <= 0: return float("nan"), float("nan")
    z = (a1 - a2) / np.sqrt(var_diff)
    from scipy.stats import norm
```

```
    return float(z), float(2*(1 - norm.cdf(abs(z))))

# ── Bayesian LR via L2 with non-zero center (penalized IRLS) ──────────
class BayesianLogReg:
    """Logistic regression with Gaussian priors β ~ N(prior_mean, prior_sd²).
    Equivalent to L2 with non-zero center: minimize □ − + 0.5 βμ(−)' Λ βμ(−)
    where Λ = diag(1/prior_sd²). Solved by Newton-Raphson.
    """
    def __init__(self, prior_means, prior_sds, max_iter=200, tol=1e-6):
        self.prior_means = np.asarray(prior_means, dtype=float)
        self.prior_sds   = np.asarray(prior_sds, dtype=float)
        self.max_iter, self.tol = max_iter, tol
    def fit(self, X, y):
        X = np.asarray(X, dtype=float); y = np.asarray(y, dtype=float)
        n, p = X.shape
        Xa = np.column_stack([np.ones(n), X])  # add intercept
        # Prior: weakly informative on intercept (sd=10), informative on coeffs
        mu = np.concatenate([[0.0], self.prior_means])
        sd = np.concatenate([[10.0], self.prior_sds])
        Lambda = np.diag(1.0 / (sd ** 2))
        beta = mu.copy()
        for _ in range(self.max_iter):
            eta = Xa @ beta
            mu_p = 1.0 / (1.0 + np.exp(-np.clip(eta, -30, 30)))
            W = mu_p * (1 - mu_p) + 1e-8
            grad = Xa.T @ (mu_p - y) + Lambda @ (beta - mu)
            H = (Xa.T * W) @ Xa + Lambda
            step = np.linalg.solve(H, grad)
            beta_new = beta - step
            if np.max(np.abs(beta_new - beta)) < self.tol:
                beta = beta_new; break
            beta = beta_new
        self.coef_ = beta[1:].reshape(1, -1)
        self.intercept_ = np.array([beta[0]])
        return self
    def predict_proba(self, X):
        X = np.asarray(X, dtype=float)
        eta = X @ self.coef_.ravel() + self.intercept_[0]
        p = 1.0 / (1.0 + np.exp(-np.clip(eta, -30, 30)))
        return np.column_stack([1 - p, p])


def derive_eicu_priors(features_bidmc):
    """Fit elastic-net LR on eICU shared features, return prior means and sds
    for BIDMC features (mapped). Unshared features get prior mean=0, sd=2.5.
    """
    print("  Fitting elastic-net LR on eICU to derive priors ...", flush=True)
    eicu = load_eicu()
    Xe = eicu[SHARED_EICU_NAMES].fillna(eicu[SHARED_EICU_NAMES].median()).astype(float)
    ye = eicu["seizure"].astype(int)
    sc = StandardScaler(); Xe_s = sc.fit_transform(Xe)
    eicu_mu, eicu_sigma = sc.mean_, sc.scale_
    lr = LogisticRegression(penalty="elasticnet", l1_ratio=0.5, C=0.1,
                            solver="saga", max_iter=5000, class_weight="balanced",
```

```
                                n_jobs=1, random_state=SEED)
    lr.fit(Xe_s, ye)
    eicu_coefs = lr.coef_[0]
... (170 more lines elided — see full script in the repository) ...
```

BayesianLogReg is a from-scratch implementation of logistic regression with Gaussian priors centred at user-supplied means. derive_eicu_priors fits an elastic-net LR on eICU's shared features and returns those coefficients as the prior means.

Firth matches the BalancedRandomForest baseline on AUC (0.681 vs 0.676; DeLong $p = 0.81$) with a 3.3-fold improvement in Brier score. The Bayesian-with-eICU-priors variant degrades discrimination significantly — a biological-mismatch warning, not a statistical failure.

7 `25_conformal_prediction.py` — turning probabilities into bedside decisions

Conformal prediction turns any classifier into a procedure that emits prediction sets with distribution-free coverage guarantees. The class-conditional variant (Mondrian) guarantees coverage separately for each true class.

```
"""Task 25 — Class-conditional (Mondrian) conformal prediction for clinical deployment.

Lit-review motivation: with n=655 / 48 events, AUC ceiling is essentially fixed
by Bernoulli noise. The decision-relevant reframing is calibrated risk
stratification with distribution-free coverage guarantees. Class-conditional
conformal prediction (Mondrian, Vovk 2003; modern: Angelopoulos & Bates 2021
arXiv:2107.07511; clinical: García-Cremades et al. PMLR 252, 2024) provides
exactly this.

We use the split-conformal scheme:
  1. Train BalancedRF on the calibration-training split
  2. On a held-out calibration set, compute nonconformity scores 1 - P(true class)
  3. Choose αq_ as the α(1-) class-conditional quantile of those scores
  4. For each test point, the prediction set is {y : 1 - P(y) ≤ αq_}

Reported metrics:
  • Empirical coverage (should ≈ 1 - α)
  • Average prediction-set size
  • Singleton-set rate (cases confidently classified)
  • Rule-out rate at α=0.10 (cases assigned the "no seizure" singleton)
  • Rule-in rate at α=0.10 (cases assigned the "seizure" singleton)

We compare three base models for the conformal procedure:
  • BalancedRandomForest
  • Diverse stack (from Task 22, if available)
  • Bayesian LR (from Task 24, if available)

Outputs:
  results/25_conformal.csv
  figures/25_conformal.{png,pdf}
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
os.environ["OMP_NUM_THREADS"] = "1"

import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score, brier_score_loss
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```python
from imblearn.ensemble import BalancedRandomForestClassifier

from _shared import (load_bidmc, POSTOP_A_FEATURES, POSTOP_B_FEATURES,
                     RES, FIG, SEED)

N_SPLITS = 5
N_REPEATS = 3
ALPHAS = [0.05, 0.10, 0.20]

def make_prep(features):
    return ColumnTransformer([
        ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                          ("sc",  StandardScaler())]), features)])

def make_brf(features):
    return Pipeline([("prep", make_prep(features)),
                     ("clf", BalancedRandomForestClassifier(
                          n_estimators=300, min_samples_leaf=2, n_jobs=1,
                          random_state=SEED))])

def class_conditional_conformal(p_cal, y_cal, p_test, alpha):
    """Mondrian conformal: separate calibration per class.
    Returns prediction-set membership: shape (n_test, 2) booleans.
    For each class c, q_c = α(1-) empirical quantile of (1 - p_c) on calibration
    examples of class c. Prediction set includes y if (1 - p_y) ≤ q_y.
    """
    p_cal = np.clip(p_cal, 1e-6, 1 - 1e-6)
    p_test = np.clip(p_test, 1e-6, 1 - 1e-6)
    # nonconformity = 1 - P(true class) on calibration set
    nc_pos = 1.0 - p_cal[y_cal == 1]      # cases where true=1
    nc_neg = 1.0 - (1.0 - p_cal[y_cal == 0])  # cases where true=0 → 1-P(y=0)
    if len(nc_pos) < 2 or len(nc_neg) < 2:
        return np.zeros((len(p_test), 2), dtype=bool)
    # finite-sample correction: (n+1)α(1-)/n th value
    n_pos = len(nc_pos); n_neg = len(nc_neg)
    q_pos = np.quantile(nc_pos, min(1.0, (1 - alpha) * (n_pos + 1) / n_pos))
    q_neg = np.quantile(nc_neg, min(1.0, (1 - alpha) * (n_neg + 1) / n_neg))
    # for each test point, include class c if its nonconformity ≤ q_c
    include_pos = (1.0 - p_test)         <= q_pos       # include "1" if (1-P(1)) ≤ q_pos
    include_neg = (1.0 - (1.0 - p_test)) <= q_neg       # include "0" if (1-P(0)) ≤ q_neg
    return np.column_stack([include_neg, include_pos]).astype(bool)


def evaluate_conformal(make_pipe_fn, features, X, y, alphas=ALPHAS,
                       n_splits=N_SPLITS, n_repeats=N_REPEATS):
    """Cross-conformal: split → fit on train, calibrate on val, predict on test.
    We use a 3-way split per fold: train (60%), calibration (20%), test (20%).
    Aggregate metrics over repeated CV.
    """
    metrics = {a: {"coverage": [], "set_size": [], "singleton_rate": [],
                   "rule_out_rate": [], "rule_in_rate": []} for a in alphas}
    rng = np.random.default_rng(SEED)
    for rep in range(n_repeats):
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
```

```
                                random_state=SEED + rep)
        for fold_idx, (trainval, test) in enumerate(skf.split(X, y)):
            # Split trainval → train (75%) + cal (25%)
            yv = y.iloc[trainval]
            pos_v = np.where(yv == 1)[0]; neg_v = np.where(yv == 0)[0]
            rng_l = np.random.default_rng(SEED + rep * 100 + fold_idx)
            rng_l.shuffle(pos_v); rng_l.shuffle(neg_v)
            cal_pos = pos_v[: max(2, len(pos_v) // 4)]
            cal_neg = neg_v[: max(2, len(neg_v) // 4)]
            cal_idx = trainval[np.concatenate([cal_pos, cal_neg])]
            train_idx = np.setdiff1d(trainval, cal_idx)
            X_tr, y_tr = X.iloc[train_idx], y.iloc[train_idx]
            X_cal, y_cal = X.iloc[cal_idx], y.iloc[cal_idx]
            X_te, y_te = X.iloc[test], y.iloc[test]
            if y_tr.sum() < 5 or y_cal.sum() < 2: continue

            try:
                pipe = make_pipe_fn()
                pipe.fit(X_tr, y_tr)
                p_cal = pipe.predict_proba(X_cal)[:, 1]
                p_te  = pipe.predict_proba(X_te)[:, 1]
            except Exception:
                continue

            for a in alphas:
                sets = class_conditional_conformal(p_cal, y_cal.values, p_te, a)
                # set membership: sets[:, 0] = "0" included; sets[:, 1] = "1" included
                set_size = sets.sum(axis=1)
                # coverage: fraction where true label is included
                cov = float(np.mean([sets[i, int(y_te.iloc[i])] for i in range(len(y_te))]))
                # singleton-rule-out: only "0" included, true class included
                rule_out = float(np.mean(sets[:, 0] & (~sets[:, 1])))
                rule_in  = float(np.mean(sets[:, 1] & (~sets[:, 0])))
                singleton = float(np.mean(set_size == 1))
                metrics[a]["coverage"].append(cov)
                metrics[a]["set_size"].append(float(set_size.mean()))
                metrics[a]["singleton_rate"].append(singleton)
                metrics[a]["rule_out_rate"].append(rule_out)
                metrics[a]["rule_in_rate"].append(rule_in)
    summary = []
... (103 more lines elided — see full script in the repository) ...
```

class_conditional_conformal computes the nonconformity score 1 − P(true class) on every calibration example, takes the $(1-\alpha)(n+1)/n$ empirical quantile separately for the positive and negative classes, and for each test point includes each class in the prediction set iff its nonconformity does not exceed that class's quantile.

8 `16_voi_evpi.py` — value-of-information analysis

Value-of-information analysis answers: of all the uncertain parameters in our decision model, which would actually change the optimal strategy if we knew them perfectly? EVPI is the population-scale upper bound on the value of any future study. EVPPI ranks per-parameter research priorities.

```
"""Task 16 — Value-of-information (EVPI / EVPPI) analysis for the CEA.

EVPI quantifies the maximum expected health-and-cost-equivalent value of
resolving all parameter uncertainty:

    EVPIλ() = θE_ [max_d NMB(d, θ; λ)] − max_d θE_ NMB(d, θ; λ)

EVPPIφ(; λ) is the analogue restricted to a single parameter subset φ, the
expected value of perfectly knowing φ. We estimate it via the Strong-Oakley
(MDM 2014) non-parametric regression approach: a smooth fit of NMB on the
focal parameter, with the fitted values approximating E[NMB | φ].

WTP base case is $100,000/QALY (Neumann 2014; Vanness 2021; Crespo 2023).
We also report EVPI at $50k and $150k.

Outputs:
  results/16_voi_evpi.csv
  results/16_voi_evppi.csv
  figures/16_voi_evpi.{png,pdf}
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
os.environ["OMP_NUM_THREADS"] = "1"

import numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy.interpolate import UnivariateSpline

# Reuse PSA helpers from the existing CEA module
from importlib.util import spec_from_file_location, module_from_spec
_HERE = os.path.dirname(__file__)
_spec = spec_from_file_location("cea_mod", os.path.join(_HERE, "10_11_cea_pairwise.py"))
cea = module_from_spec(_spec); _spec.loader.exec_module(cea)

from _shared import RES, FIG, SEED

# ── Annual incident cSDH-surgery population (Move 3 lit input) ──────────
ANNUAL_POPULATION = 40_000      # midpoint of 30–50k cSDH operations / yr US
DECISION_HORIZON  = 10
DISCOUNT_RATE     = 0.03
WTPs              = (50_000, 100_000, 150_000)
N_PSA_VOI         = 5_000       # 5000 iterations — enough for stable EVPI

STRATEGIES = ["obs", "aed", "mla", "mlg"]   # column suffixes in PSA frame


```

```python
def run_psa_tracked(n=N_PSA_VOI, seed=SEED):
    """Re-run the existing PSA but also persist the sampled parameters so
    we can fit per-parameter EVPPI regressions."""
    rng = np.random.default_rng(seed)
    a_prev, b_prev = cea.beta_params_from_ci(0.12, 0.05, 0.20)
    a_sens, b_sens = cea.beta_params_from_ci(0.842, 0.722, 0.918)
    a_spec, b_spec = cea.beta_params_from_ci(0.504, 0.439, 0.569)
    a_pse,  b_pse  = cea.beta_params_from_ci(0.10, 0.05, 0.18)
    a_rrr,  b_rrr  = cea.beta_params_from_ci(0.45, 0.25, 0.65)
    a_pe_det, b_pe_det = cea.beta_params_from_ci(0.03, 0.01, 0.06)
    a_pe_und, b_pe_und = cea.beta_params_from_ci(0.12, 0.05, 0.22)
    a_pdet_ceeg, b_pdet_ceeg = cea.beta_params_from_ci(0.95, 0.85, 0.99)
    a_pdet_clin, b_pdet_clin = cea.beta_params_from_ci(0.30, 0.15, 0.50)
    sh_aed_drug, sc_aed_drug = cea.gamma_params(200.0, 50)
    sh_aed_adv, sc_aed_adv   = cea.gamma_params(1507.0, 500)
    sh_aed_cont, sc_aed_cont = cea.gamma_params(441.0, 150)
    sh_aed_adv_c, sc_aed_adv_c   = cea.gamma_params(1293.0, 400)
    sh_aed_cont_c, sc_aed_cont_c = cea.gamma_params(140.0, 50)
    sh_se_early, sc_se_early = cea.gamma_params(13200.0, 4000)
    sh_se_late,  sc_se_late  = cea.gamma_params(35000.0, 10000)
    sh_ceeg, sc_ceeg = cea.gamma_params(1500.0, 500)
    sh_hosp, sc_hosp = cea.gamma_params(3500.0, 700)
    sh_icu,  sc_icu  = cea.gamma_params(5000.0, 1500)
    sh_szwu, sc_szwu = cea.gamma_params(2500.0, 800)
    sh_epi,  sc_epi  = cea.gamma_params(15000.0, 5000)

    rows = []
    p_global = cea.params_global
    for i in range(n):
        ps = dict(
            p_sz     = rng.beta(a_prev, b_prev),
            sens     = rng.beta(a_sens, b_sens),
            spec     = rng.beta(a_spec, b_spec),
            p_se     = rng.beta(a_pse,  b_pse),
            rrr      = rng.beta(a_rrr,  b_rrr),
            pe_d     = rng.beta(a_pe_det, b_pe_det),
            pe_u     = rng.beta(a_pe_und, b_pe_und),
            pd_ceeg  = rng.beta(a_pdet_ceeg, b_pdet_ceeg),
            pd_clin  = rng.beta(a_pdet_clin, b_pdet_clin),
            c_aed_d  = rng.gamma(sh_aed_drug, sc_aed_drug),
            c_aed_a  = rng.gamma(sh_aed_adv,  sc_aed_adv),
            c_aed_c  = rng.gamma(sh_aed_cont, sc_aed_cont),
            aed_d    = rng.uniform(14, 120),
            c_aed_ac = rng.gamma(sh_aed_adv_c,  sc_aed_adv_c),
            c_aed_cc = rng.gamma(sh_aed_cont_c, sc_aed_cont_c),
            aed_dc   = rng.uniform(7, 42),
            c_se_e   = rng.gamma(sh_se_early, sc_se_early),
            c_se_l   = rng.gamma(sh_se_late,  sc_se_late),
            icu_red  = rng.uniform(0.3, 1.5),
            c_ceeg   = rng.gamma(sh_ceeg, sc_ceeg),
            c_hosp   = rng.gamma(sh_hosp, sc_hosp),
            c_icu    = rng.gamma(sh_icu,  sc_icu),
            c_szwu   = rng.gamma(sh_szwu, sc_szwu),
            c_epi    = rng.gamma(sh_epi,  sc_epi),
```

```
            u_dec    = rng.uniform(0.05, 0.20),
            p_rec    = rng.beta(3, 17),
            pd_b     = rng.beta(2.5, 97.5),
            pd_e     = rng.beta(2, 98),
            ceeg_d   = float(rng.choice([2, 3, 4, 5], p=[0.2, 0.4, 0.3, 0.1])),
        )
        shared = dict(
            p_seizure_base=ps["p_sz"], aed_rrr=ps["rrr"],
            cost_aed_drug=ps["c_aed_d"], cost_aed_adverse=ps["c_aed_a"],
            cost_aed_continuation=ps["c_aed_c"], expected_aed_days=ps["aed_d"],
            cost_aed_adverse_ceeg=ps["c_aed_ac"],
            cost_aed_continuation_ceeg=ps["c_aed_cc"],
            expected_aed_days_ceeg=ps["aed_dc"],
            cost_se_early=ps["c_se_e"], cost_se_late=ps["c_se_l"],
            icu_los_reduction=ps["icu_red"],
            p_detect_ceeg=ps["pd_ceeg"], p_detect_clinical=ps["pd_clin"],
            cost_ceeg=ps["c_ceeg"], cost_hosp_day=ps["c_hosp"], cost_icu=ps["c_icu"],
            cost_sz_workup=ps["c_szwu"], p_se=ps["p_se"],
            utility_sz_decrement=ps["u_dec"], p_recurrence=ps["p_rec"],
            p_epilepsy_detected=ps["pe_d"], p_epilepsy_undetected=ps["pe_u"],
            p_death_base=ps["pd_b"], p_death_excess=ps["pd_e"],
            cost_annual_epi=ps["c_epi"], ceeg_days=ps["ceeg_d"],
        )
        ml_shared = dict(sens=ps["sens"], spec=ps["spec"], **shared)
        c_o, q_o = cea.run_strategy("observation", p_global, **shared)
        c_a, q_a = cea.run_strategy("universal_aed", p_global, **shared)
        c_ma, q_ma = cea.run_strategy("ml_aed_only", p_global, **ml_shared)
        c_mg, q_mg = cea.run_strategy("ml_guided", p_global, **ml_shared)
        row = dict(ps)
        row.update({
            "cost_obs": c_o, "qaly_obs": q_o,
            "cost_aed": c_a, "qaly_aed": q_a,
            "cost_mla": c_ma, "qaly_mla": q_ma,
            "cost_mlg": c_mg, "qaly_mlg": q_mg,
        })
        rows.append(row)
        if (i + 1) % 500 == 0:
            print(f"  PSA {i+1:>5d}/{n}", flush=True)
    return pd.DataFrame(rows)
... (134 more lines elided — see full script in the repository) ...
```

compute_evpi is the population-EVPI formula. evppi_strong_oakley implements the Strong-Oakley non-parametric regression approximation per parameter and feeds the tornado chart in Figure 6.

9 29_main_figures_jnnp.py — publication-grade figures

Every main paper figure is built by a dedicated function (figure_1 …figure_6) with a consistent house style: Helvetica sans-serif, open-box axes, bold uppercase panel labels, a restrained CMYK-safe palette, 300-dpi PNG and editable vector PDF output.

```
"""Task 29 — Main paper figures, JNNP-style.

JNNP aesthetic conventions (consistent with the journal's published figures):
  • Helvetica / Arial sans-serif throughout (DejaVu Sans fallback)
  • Open-box axes: only bottom and left spines visible; tick direction out
  • Bold uppercase panel labels (A, B, C) in the top-left of each panel
  • Restrained palette: navy / rust / forest-green / muted-grey
  • Light grey y-axis grid only, alpha 0.30
  • 300 dpi PNG (print) and vector PDF (for figure submission)
  • CMYK-safe primary colours; usable in grayscale conversion

Rebuilds —F1F6 as native matplotlib figures (no PIL image-wrapping),
sourced directly from the result CSVs and the existing scripts' data.

Outputs:
  figures/F1_discrimination.{png,pdf}
  figures/F2_calibration_dca.{png,pdf}
  figures/F3_method_battery.{png,pdf}
  figures/F4_conformal.{png,pdf}
  figures/F5_cea.{png,pdf}
  figures/F6_voi.{png,pdf}
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
os.environ["OMP_NUM_THREADS"] = "1"

import numpy as np, pandas as pd
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
import matplotlib.patches as mpatches
import json

from _shared import RES, FIG, CACHE

# ── JNNP style ──────────────────────────────────────────────
# Prefer Helvetica / Arial; fall back to a sans-serif that ships with matplotlib.
_pref_fonts = ["Helvetica", "Helvetica Neue", "Arial", "Liberation Sans",
               "DejaVu Sans"]
_available = {f.name for f in font_manager.fontManager.ttflist}
JNNP_FONT = next((f for f in _pref_fonts if f in _available), "DejaVu Sans")

plt.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": [JNNP_FONT],
    "font.size": 9,
    "axes.titlesize": 10,
```

```
    "axes.titleweight": "bold",
    "axes.labelsize": 9,
    "axes.labelweight": "regular",
    "axes.edgecolor": "#222222",
    "axes.linewidth": 0.8,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "axes.axisbelow": True,
    "grid.color": "#cccccc",
    "grid.linewidth": 0.5,
    "grid.alpha": 0.45,
    "xtick.direction": "out",
    "ytick.direction": "out",
    "xtick.major.size": 4,
    "ytick.major.size": 4,
    "xtick.minor.size": 2,
    "ytick.minor.size": 2,
    "xtick.major.width": 0.7,
    "ytick.major.width": 0.7,
    "xtick.color": "#222222",
    "ytick.color": "#222222",
    "xtick.labelsize": 8,
    "ytick.labelsize": 8,
    "legend.fontsize": 8,
    "legend.frameon": False,
    "legend.handlelength": 1.5,
    "legend.handletextpad": 0.6,
    "figure.dpi": 110,
    "savefig.dpi": 300,
    "savefig.bbox": "tight",
    "savefig.pad_inches": 0.05,
    "pdf.fonttype": 42,          # editable text in PDF
    "ps.fonttype": 42,
})

# JNNP-safe palette (CMYK-friendly, distinct in grayscale)
COL = {
    "navy":    "#1F3D5C",
    "rust":    "#B5532C",
    "forest":  "#2E6B45",
    "indigo":  "#5C4B8A",
    "ochre":   "#B58A2E",
    "slate":   "#4A5560",
    "soft":    "#9FB6C9",
    "rule":    "#000000",
    "grey":    "#6E6E6E",
    "highlight":"#D03C29",
}

def add_panel_label(ax, label, *, offset=(-0.12, 1.04)):
    ax.text(offset[0], offset[1], label,
            transform=ax.transAxes,
            fontsize=11, fontweight="bold",
```

```
            va="bottom", ha="left")

def style_axis(ax, *, ygrid=True, xgrid=False):
    ax.tick_params(axis="both", which="major")
    if ygrid:
        ax.yaxis.grid(True, color=COL["soft"], alpha=0.30, linewidth=0.4)
    if xgrid:
        ax.xaxis.grid(True, color=COL["soft"], alpha=0.30, linewidth=0.4)
    else:
... (512 more lines elided — see full script in the repository) ...
```

The plt.rcParams.update block sets the house style once; every panel inherits from it. add_panel_label puts the bold A/B/C labels; style_axis applies the tick direction; figure_legend_below places a single shared legend beneath the figure so legends never overlap data.

10 `27_build_jnnp_manuscript.py` — building the Word manuscript from code

This script writes the manuscript .docx file end-to-end using python-docx. Building the manuscript programmatically means every numerical claim in the text is traceable to the results CSV that produced it, and every version of the manuscript is a deterministic function of the underlying analysis.

```
"""Task 27 — Journal-format manuscript build (proof-of-concept framing).

Target: a high-impact neurology/neurosurgery journal accepting Original
Research articles with the following conventional limits (most BMJ-family
and similar journals):
  • Original Research: 4,000 words main text
  • Structured abstract: 250 words
  • Maximum 6 figures/tables in main text
  • Vancouver references (numbered, superscript)
  • TRIPOD-AI reporting checklist for prediction models
  • Supplementary material accepted separately

Outputs:
  manuscript/main_manuscript.docx     ≤(4000 words, —F1F6, Table 1)
  manuscript/supplementary.docx       (Figures —S1S7, Tables —S1S5,
                                       TRIPOD-AI checklist)
"""
import sys, os
sys.path.insert(0, os.path.dirname(__file__))
import pandas as pd

from docx import Document
from docx.shared import Inches, Pt, RGBColor
from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_LINE_SPACING
from docx.oxml.ns import qn
from docx.oxml import OxmlElement

from _shared import OUT, RES, FIG

MANUS_DIR = OUT / "manuscript"
MAIN_PATH = MANUS_DIR / "main_manuscript.docx"
SUPP_PATH = MANUS_DIR / "supplementary.docx"

# ── Doc helpers ──────────────────────────────────────────────
def add_heading(doc, text, level=1):
    h = doc.add_heading(text, level=level)
    for r in h.runs:
        r.font.color.rgb = RGBColor(0x14, 0x1F, 0x3A)
        r.font.name = "Times New Roman"
    return h

def add_para(doc, text, *, italic=False, bold=False, indent=False, size=11,
             alignment=WD_ALIGN_PARAGRAPH.JUSTIFY):
    p = doc.add_paragraph()
    p.alignment = alignment
    p.paragraph_format.line_spacing_rule = WD_LINE_SPACING.ONE_POINT_FIVE
```

```
    p.paragraph_format.space_after = Pt(6)
    if indent: p.paragraph_format.first_line_indent = Inches(0.3)
    r = p.add_run(text)
    r.font.size = Pt(size); r.font.name = "Times New Roman"
    r.italic = italic; r.bold = bold
    return p

def add_runs(doc, runs, *, indent=True, alignment=WD_ALIGN_PARAGRAPH.JUSTIFY):
    """runs = list of (text, dict-of-formatting)."""
    p = doc.add_paragraph()
    p.alignment = alignment
    p.paragraph_format.line_spacing_rule = WD_LINE_SPACING.ONE_POINT_FIVE
    p.paragraph_format.space_after = Pt(6)
    if indent: p.paragraph_format.first_line_indent = Inches(0.3)
    for text, fmt in runs:
        r = p.add_run(text)
        r.font.size = Pt(fmt.get("size", 11)); r.font.name = "Times New Roman"
        r.bold = fmt.get("bold", False)
        r.italic = fmt.get("italic", False)
        if fmt.get("superscript"):
            r.font.superscript = True
    return p

def add_figure(doc, path, *, caption, width=Inches(6.5)):
    if not os.path.exists(path):
        add_para(doc, f"[Figure missing: {path}]", italic=True); return
    p = doc.add_paragraph(); p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    p.add_run().add_picture(str(path), width=width)
    cp = doc.add_paragraph(); cp.alignment = WD_ALIGN_PARAGRAPH.LEFT
    cr = cp.add_run(caption)
    cr.font.size = Pt(10); cr.italic = True; cr.font.name = "Times New Roman"

def add_table_from_df(doc, df, caption=None):
    if caption:
... (744 more lines elided — see full script in the repository) ...
```

The structure mirrors a normal Word workflow: setup_document sets page margins; add_heading, add_para, add_runs handle text; register_figure and register_table accumulate figures and tables; render_collected_tables_and_figures emits them all in a single Tables-and-Figures section at the end.

11 `30_export_calculator_assets.py` — packaging the model for the dashboard

The browser-side calculator cannot run scikit-learn — it is JavaScript. To let the browser compute the same prediction the Python model would, this script extracts the deployment-ready coefficients, the feature scaler statistics, and the conformal quantiles and writes them to site/model_assets.json.

```
"""Task 30 — Export deployment-ready assets for the web calculator.

Produces:
  github_repo/site/model_assets.json
      • Firth penalized LR coefficients fit on the FULL BIDMC cohort
      • Feature means / SDs for per-feature z-scaling
      • Class-conditional conformal nonconformity quantiles at α ∈ {0.05, 0.10, 0.20}
      • CEA base-case parameters (cost / QALY per strategy)
      • Population-savings template (per-patient ΔCost, ΔQALY versus
        observation for ML-AED and ML-cEEG)

The web calculator at site/calculator.html consumes this JSON to produce
a deterministic prediction without any patient data leaving the browser.
"""
import sys, os, json
sys.path.insert(0, os.path.dirname(__file__))
os.environ["OMP_NUM_THREADS"] = "1"

import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold
from firthlogist import FirthLogisticRegression

from _shared import load_bidmc, POSTOP_A_FEATURES, OUT, SEED

SITE_DIR = OUT / "github_repo" / "site"
SITE_DIR.mkdir(parents=True, exist_ok=True)


def main():
    df = load_bidmc()
    y = df["seizure"].astype(int).values
    X_raw = df[POSTOP_A_FEATURES].copy()

    imp = SimpleImputer(strategy="median")
    X_imp = pd.DataFrame(imp.fit_transform(X_raw), columns=POSTOP_A_FEATURES)
    sc = StandardScaler()
    X_z = sc.fit_transform(X_imp)

    # Firth on full data — for deployment coefficients
    print(f"Fitting Firth penalized LR on full BIDMC (n={len(y)}, events={int(y.sum())})...",
          flush=True)
    firth = FirthLogisticRegression(max_iter=200, max_halfstep=25, tol=1e-6,
```

```
                                    skip_pvals=True, skip_ci=True, wald=True)
    firth.fit(X_z, y)
    p_full = firth.predict_proba(X_z)[:, 1]
    intercept = float(firth.intercept_)
    coefs = firth.coef_.tolist()
    print(f"  intercept = {intercept:.4f}")
    for f, c in zip(POSTOP_A_FEATURES, coefs):
        print(f"  β[{f:<22s}] = {c:+.4f}")

    # ── Class-conditional conformal quantiles ──
    # Use the same split as the conformal evaluation (5-fold, fit on 75% +
    # calibrate on 25%) and aggregate the nonconformity scores
    print("\nComputing class-conditional conformal nonconformity quantiles...",
          flush=True)
    nc_pos_all, nc_neg_all = [], []
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
    for tr, te in skf.split(X_z, y):
        # split trainval -> train (75%) + cal (25%)
        rng = np.random.default_rng(SEED)
        pos_tr = np.where(y[tr] == 1)[0]; neg_tr = np.where(y[tr] == 0)[0]
        rng.shuffle(pos_tr); rng.shuffle(neg_tr)
        cal_pos = tr[pos_tr[: max(2, len(pos_tr) // 4)]]
        cal_neg = tr[neg_tr[: max(2, len(neg_tr) // 4)]]
        cal_idx = np.concatenate([cal_pos, cal_neg])
        train_idx = np.setdiff1d(tr, cal_idx)
        if y[train_idx].sum() < 5: continue
        m = FirthLogisticRegression(max_iter=200, max_halfstep=25, tol=1e-6,
                                    skip_pvals=True, skip_ci=True, wald=True)
        m.fit(X_z[train_idx], y[train_idx])
        p_cal = np.clip(m.predict_proba(X_z[cal_idx])[:, 1], 1e-6, 1 - 1e-6)
        y_cal = y[cal_idx]
        nc_pos = (1.0 - p_cal[y_cal == 1])
        nc_neg = (1.0 - (1.0 - p_cal[y_cal == 0]))
        nc_pos_all.extend(nc_pos.tolist())
        nc_neg_all.extend(nc_neg.tolist())
    nc_pos_all = np.array(nc_pos_all); nc_neg_all = np.array(nc_neg_all)
    alphas = [0.05, 0.10, 0.20]
    q_pos = {f"alpha_{a:.2f}": float(np.quantile(nc_pos_all, 1 - a)) for a in alphas}
    q_neg = {f"alpha_{a:.2f}": float(np.quantile(nc_neg_all, 1 - a)) for a in alphas}
    print(f"  positive-class nonconformity quantiles: {q_pos}")
    print(f"  negative-class nonconformity quantiles: {q_neg}")

    # ── CEA base-case (per-patient expected outcomes) ──
    # values from scripts/14_decision_tree.py rollback
    cea_rollback = pd.read_csv(OUT / "results" / "14_decision_tree_rollback.csv")
    cea_dict = cea_rollback.set_index("strategy")[["E_cost_USD", "E_QALY"]].to_dict()

    # ── Package and write ──
    payload = {
        "model": {
            "type": "Firth penalized logistic regression",
            "fit_on": "BIDMC postoperative-A, n=655, 48 events",
            "intercept": intercept,
            "features": POSTOP_A_FEATURES,
```

```
            "coefficients": coefs,
            "feature_means": sc.mean_.tolist(),
            "feature_sds":   sc.scale_.tolist(),
            "feature_medians_for_imputation": imp.statistics_.tolist(),
        },
        "conformal": {
            "method": "class-conditional (Mondrian) split conformal",
            "calibration_split": "75% train / 25% calibration, 5-fold pooled",
            "nonconformity_quantiles_positive": q_pos,
            "nonconformity_quantiles_negative": q_neg,
        },
        "cea": {
            "wtp_threshold_usd_per_qaly": 100000,
            "discount_rate": 0.03,
            "horizon_years": 10,
            "strategies": cea_dict,
        },
        "metadata": {
            "n_features": len(POSTOP_A_FEATURES),
            "n_patients": int(len(y)),
... (14 more lines elided — see full script in the repository) ...
```

The Firth model is fit on the full BIDMC cohort. The class-conditional conformal quantiles are computed by repeated 5-fold splits with 75/25 train/calibration partitioning; the $(1-\alpha)(n+1)/n$ quantile of each class is taken. Everything is bundled into a single JSON file.

12 The interactive dashboards — HTML, CSS, JavaScript

The three dashboards on the companion site are static HTML pages served by GitHub Pages. Each page bundles its own JavaScript so all computation happens in the user's browser.

12.1 Anatomy of an HTML page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Risk calculator</title>
  <style> /* CSS — visual styling */ </style>
</head>
<body>
  <header>...</header>
  <main>...</main>
  <script> /* JavaScript — behaviour */ </script>
</body>
</html>
```

HTML defines structure; CSS defines visual style; JavaScript defines behaviour.

12.2 How the risk calculator works

On page load the calculator fetches model_assets.json, extracts the Firth coefficients and the conformal quantiles, and attaches an event listener to every form input. Whenever the user changes any input, the recompute function fires: it reads all 21 feature values, z-scales them using the means and SDs from the JSON, computes the dot product with the coefficients, adds the intercept, applies the sigmoid, and produces a probability.

```
function recompute() {
  if (!MODEL) return;
  // Build raw vector in feature order
  const raw = FIRTH_FEATURES_ORDER.map(f => getInputValue(f));
  const means = MODEL.model.feature_means;
  const sds   = MODEL.model.feature_sds;
  const coefs = MODEL.model.coefficients;
  const intercept = MODEL.model.intercept;
  // z-scale
  let z = 0;
  for (let i = 0; i < raw.length; i++) {
    const v = (raw[i] - means[i]) / sds[i];
    z += v * coefs[i];
  }
  z += intercept;
  const p = 1 / (1 + Math.exp(-z));
  document.getElementById("prob").textContent = (p * 100).toFixed(1) + "%";
  document.getElementById("prob-meta").textContent =
    "Logit = " + z.toFixed(3) + " · ratio vs prevalence (7.3%): " +
    (p / 0.073).toFixed(2) + "×";
```

```
21
22    // Conformal sets
23    const alpha = parseFloat(document.getElementById("alpha").value);
24    const aKey = "alpha_" + alpha.toFixed(2);
25    const qPos = MODEL.conformal.nonconformity_quantiles_positive[aKey];
26    const qNeg = MODEL.conformal.nonconformity_quantiles_negative[aKey];
27    const incPos = (1 - p) <= qPos;       // include "seizure"
28    const incNeg = (1 - (1 - p)) <= qNeg; // include "no seizure"  i.e.  p <= qNeg
29    let setLabel, recClass, recText;
30    if (incPos && !incNeg) {
31      setLabel = "{ seizure }";
32      recClass = "rulein";
33      recText = "RULE-IN. At the selected confidence level, this patient is "
34               + "classified as high-risk for postoperative seizure. Consider "
35               + "AED prophylaxis and/or continuous EEG monitoring.";
36    } else if (!incPos && incNeg) {
37      setLabel = "{ no seizure }";
38      recClass = "ruleout";
39      recText = "RULE-OUT. At the selected confidence level, postoperative "
40               + "seizure can be confidently ruled out. AED prophylaxis may "
41               + "be safely omitted, with routine clinical observation.";
42    } else if (incPos && incNeg) {
43      setLabel = "{ seizure, no seizure }";
44      recClass = "indeterminate";
45      recText = "INDETERMINATE. The model cannot provide a confident "
46               + "singleton prediction at this α. Clinical judgment should "
47               + "guide the AED-vs-monitoring decision.";
48    } else {
49      setLabel = "{ ∅ }";
50      recClass = "indeterminate";
51      recText = "NULL SET. Neither class is supported at this α — interpret "
52               + "as an outlier relative to the calibration set.";
53    }
54    const ps = document.getElementById("pred-set");
55    ps.textContent = setLabel;
56  ... (17 more lines elided — see full script in the repository) ...
```

The dot product is a single for loop. Each iteration takes one raw feature value, z-scales it, multiplies by the corresponding Firth coefficient, and accumulates into z. After the loop, the intercept is added and the sigmoid applied: p = 1/(1+exp(-z)). That gives the predicted probability. The conformal logic that follows checks whether 1-p is below each class-conditional quantile; the answer determines the prediction set and the recommendation.

12.3 How the savings calculator works

The savings calculator's JavaScript multiplies the per-patient ΔCost and ΔQALY values by the user-specified cohort size, applies the chosen discount rate over the chosen horizon, and presents the result as KPI cards and a net-monetary-benefit bar chart. No external libraries are used — the bars are styled elements whose width is set proportionally.

12.4 How the interactive callgraph works

The callgraph dashboard uses one external library — vis-network — loaded from a CDN. On page load the script fetches callgraph.json, maps each script to a node with category-coloured fill, attaches the per-node function inventory as a hidden property, and renders the network with a force-directed layout.

13 The callgraph — how the scripts depend on each other

Every analysis script in the repository imports _shared.py. Many also import each other. The callgraph is generated by parsing each .py file's Abstract Syntax Tree (Python's machine-readable representation of its own source) and extracting the import statements and top-level function definitions.

Before you modify any function in `_shared.py`, look at the callgraph and ask yourself: which scripts import this function? Any script with an arrow pointing into `_shared.py` depends on its behaviour. Test downstream before publishing the change.

14 Other scripts — what they do, in one paragraph each

14.1 `03_dca.py`

Decision-curve analysis. Reads cached out-of-fold predictions, computes Vickers net benefit at every probability threshold from 0 to 30 percent, plots the model curve against treat-all and treat-none, and writes the summary CSV used by Figure 2 panel B.

14.2 `05_temporal_leakage.py`

Temporal-leakage audit. Excludes the 24-hour and 48-hour rolling lab/vital features and the prophylactic-AED indicator, refits, and reports the resulting AUC.

14.3 `06_overfitting.py`

Overfitting diagnostics. Repeats the BalancedRandomForest fit with nested cross-validation, compares variable-importance ranks across folds (Spearman rho across 25 folds = 0.98), and produces a learning-curve plot.

14.4 `07_missing_data.py`

Missing-data sensitivity. Runs Little's MCAR test, computes the missingness-versus-outcome chi-square per feature, and pools AUC estimates across ten multiply-imputed datasets via Rubin's rules.

14.5 `08_eicu_cohort.py`

eICU cohort definition sensitivity. Fits the model in each of four prespecified cohort strata and reports bootstrap 95% AUC CIs.

14.6 `09_competing_risks.py`

Cause-specific Cox model plus IPCW Fine-Gray subdistribution-hazard model. Reports concordance indices and the Grambsch-Therneau Schoenfeld test.

14.7 `10_11_cea_pairwise.py`

Cost-effectiveness analysis with 10,000-iteration PSA. Defines the Params dataclass; samples each parameter from its own beta or gamma distribution; runs each of the four strategies; reports the cost-effectiveness plane and the acceptability curve.

14.8 `12_nis_seizure_reclassify.py`

The NIS outcome-correction analysis. Distinguishes acute symptomatic seizure from pre-existing epilepsy and reports both definitions' AUC.

14.9 `13_nis_grouped_lasso.py`

Group-LASSO with lambda-path tuning on the NIS chronic+surgical cohort under the corrected outcome.

14.10 15_radiology_nlp.py

Regex-pattern radiology NLP extractor with a synthetic validation set. Released as a tool; not promoted to a paper contribution.

14.11 17_build_slides.py

Builds the 15-minute oral-presentation deck using python-pptx.

14.12 18_bidmc_optimize.py

BIDMC model-optimization sweep. Optuna for 40 trials each on XGBoost and LightGBM; stacking ensemble; DeLong tests against the baseline.

14.13 19_transfer_learning.py

eICU to BIDMC transfer-learning experiment. Trains an XGBoost on the seven shared features, scores BIDMC patients, and uses the score as an additional feature in the BIDMC model.

14.14 20_build_manuscript.py

An earlier manuscript-build script. Superseded by 27.

14.15 21_imbalance_sweep.py

Eleven-method class-imbalance sweep on BIDMC.

14.16 22_diverse_stacking.py

Diverse-base stacking ensemble (LR + BalancedRF + XGBoost + KNN + RBF-SVM with a logistic meta-learner).

14.17 23_tabpfn_eval.py

Attempted TabPFN v2 evaluation. Reports the API-credential requirement and falls back.

14.18 26_main_figures.py

Earlier figure-builder that composes existing PNGs via PIL. Superseded by 29.

14.19 28_make_callgraph.py

Parses every .py file via ast, emits Markdown + Mermaid callgraph.

14.20 31_export_callgraph_json.py

Same parsing pipeline as 28, emits JSON for the interactive callgraph dashboard.

14.21 32_build_code_companion.py

Builds the Word version of this document.

14.22 `33_build_code_companion_pdf.py`

This script — the one that built the PDF you are reading.

15 Further study — Claude skills that help you learn the code

Claude ships with a library of task-focused expert prompts called skills, each invoked by typing the skill name with a leading slash. The following skills are particularly relevant to studying this codebase.

`/claude-code-guide`

General Python idioms, scikit-learn patterns, the Anthropic API.

`/code-review-excellence`

Structured second-pair-of-eyes review with prioritised feedback.

`/documentation-writer`

Docstrings, README sections, function-level comments (Diataxis).

`/python-project-structure`

Why a Python project is organised the way it is.

`/scientific-writing`

Translating analysis results into manuscript prose.

`/scientific-critical-thinking`

Interrogating methodological choices and bias diagnostics.

`/visualization-best-practices`

Refining figures, colour-blind safety, annotation placement.

`/statistical-analysis`

Choosing statistical tests, assumption checking, reporting.

`/the-humanizer`

Removing AI-pattern phrasing from a draft.

`/peer-review`

Structured manuscript review applying CONSORT or STROBE or TRIPOD.

`/literature-review`

Comprehensive systematically-searched literature review.

`/exploratory-data-analysis`

Initial inspection of a new dataset with quality metrics.

`/scikit-learn`

Specific questions about scikit-learn pipelines and CV.

`/statsmodels`

Formal frequentist inference beyond scikit-learn.

`/pymc`

Fitting Bayesian models with MCMC.

`/improve-codebase-architecture`

Refactoring after reading; reducing duplication.

(1) Read Chapter 1 of this document to learn the Python constructs you will see. (2) Read `_shared.py` with Chapter 2 of this document open beside you. (3) Read `02_calibration.py` and `24_firth_bayes_lr.py` with Chapters 3 and 6 open. (4) Open the calculator dashboard in your browser, then come back to Chapter 12 to see how the JavaScript implements the same model. By that point the rest of the repository will feel familiar.